Planning and Reinforcement Learning in AI Systems

(PARLAI)

Lesson 3b - Supervised Learning as Decision Making

Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

Supervised Learning

Learning from Data

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

A central goal of machine learning is to automatically discover patterns from data. Instead of manually programming rules, we provide the system with data and allow it to learn a model.

Examples of tasks that can be learned from data:

- Recognizing objects in images
- Predicting house prices
- Detecting anomalies in sensor readings
- Classifying medical images

Different learning paradigms exist depending on the type of data and feedback available.

Supervised Learning

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Supervised learning is a paradigm in machine learning where a model is trained using labeled examples.

It is one of the most common learning paradigms (and the core of most foundation models).

Supervised learning answers the question:

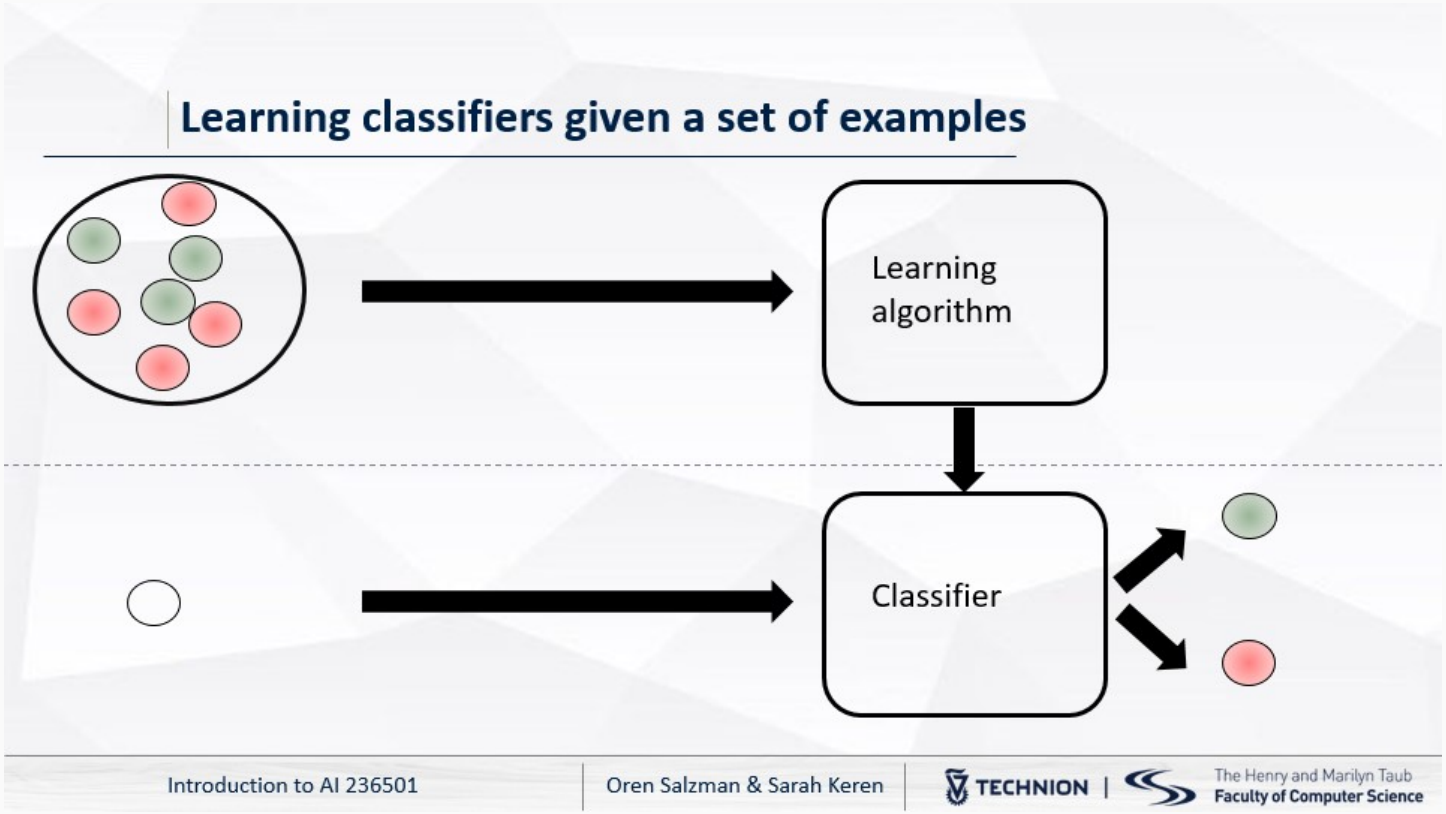
Assuming an unknown true distribution $\mathcal{D}$, how can we learn to make predictions from labeled examples sampled from $\mathcal{D}$?

Supervised Learning Examples

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

- Recognizing objects in images **using images in which objects were identified**.
- Predicting house prices **based on examples of house prices under certain conditions**
- Detecting anomalies in sensor readings **based on reading for which the actual values are known**
- Classifying medical images **based on labels provided by human doctors**

Supervised Learning in IAI



In IAI, we used supervised learning to learn the underlying model of the environment.
How is this related to computing policies and to decision-making?

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Supervised Learning Components

A **distribution** $\mathcal{D}$, from which samples are drawn i.i.d. (independent and identically distributed)

A **sample**, or labeled example (x_i, y_i) is a pair consisting of:

- an **input** (feature vector) x_i
- the corresponding **label** or target value y_i

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Supervised Learning Components

A **distribution** $\mathcal{D}$, from which samples are drawn i.i.d. (independent and identically distributed)

A **sample**, or labeled example (x_i, y_i) is a pair consisting of:

- an **input** (feature vector) x_i
- the corresponding **label** or target value y_i

A **sample set**: $S = \{(x_i, y_i)\}_{i=1}^{m}$ is used to learn a **predictor** $h(x) \in H$ from a **hypothesis class** H that generalizes well with respect to the underlying distribution $\mathcal{D}$, i.e., predicts the correct label for new inputs $(x, y) \sim \mathcal{D}$ with high probability.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Supervised Learning Components

A **distribution** $\mathcal{D}$, from which samples are drawn i.i.d. (independent and identically distributed)

A **sample**, or labeled example (x_i, y_i) is a pair consisting of:

- an **input** (feature vector) x_i
- the corresponding **label** or target value y_i

A **sample set**: $S = \{(x_i, y_i)\}_{i=1}^{m}$ is used to learn a **predictor** $h(x) \in H$ from a **hypothesis class** H that generalizes well with respect to the underlying distribution $\mathcal{D}$, i.e., predicts the correct label for new inputs $(x, y) \sim \mathcal{D}$ with high probability.

When the label y takes values in a **discrete set** (e.g., $\{0, 1\}$ or a set of classes), the task is called **classification**.

When the label y takes values in a **continuous space** (e.g., $y \in \mathbb{R}$), the task is called **regression**.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Supervised Learning Setup: Example

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

A sample includes information x_i about a person (age, gender, city of birth, ...) and her height y_i.

$$x_1 = \{35, F, NY\}, \quad y_1 = 1.58$$

$$x_2 = \{5, M, TLV\}, \quad y_2 = 0.7$$

We seek a predictor $h(x) \in H$ among a hypothesis class H, e.g., linear functions, decision trees, neural networks.

For example, we may seek a linear model

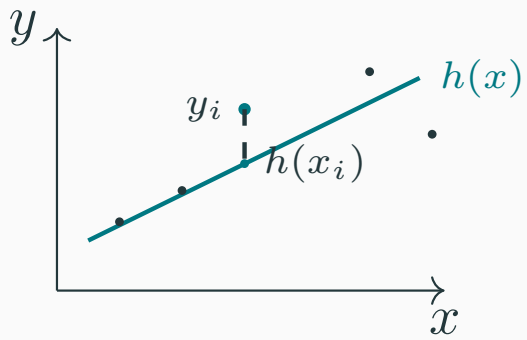
$$H = \left\{ h_w(x) = w^\top x + b \;\middle|\; w \in \mathbb{R}^d,\ b \in \mathbb{R} \right\}$$

where w are the weights and b is the bias.

Loss and Risk

In supervised learning, we aim to **learn a hypothesis that minimizes prediction error on unseen data**.

Loss $\ell(h(x), y)$: point-wise error on a **single example**



Expected Risk $\mathcal{R}(h)$: expected loss over the **data distribution**

$$\mathcal{R}(h) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[\ell(h(x), y)]$$

Since we typically do not have access to the true distribution $\mathcal{D}$, we rely instead on a **finite dataset** $(x, y) \sim \mathcal{D}$ to compute the **Empirical Risk** $\hat{\mathcal{R}}(h)$, i.e., the average loss over the **training data**.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Empirical Risk Minimization

Given a sample set $S = \{(x_i, y_i)\}_{i=1}^m$ and a hypothesis class H, we evaluate each hypothesis by its average loss on the training data.

For a hypothesis $h \in H$, the **empirical risk** is

$$\hat{\mathcal{R}}(h) = \frac{1}{m} \sum_{i=1}^{m} \ell(h(x_i), y_i)$$

The learning problem is to find a hypothesis with minimal empirical risk:

$$\hat{h} = \arg\min_{h \in H} \hat{\mathcal{R}}(h) = \arg\min_{h \in H} \frac{1}{m} \sum_{i=1}^{m} \ell(h \in h(x_i), y_i)$$

Interpretation: choose the predictor whose predictions incur the smallest average loss on the observed examples.

This principle is called **Empirical Risk Minimization (ERM)**.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Examples of Loss Functions

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Classification

- 0-1 Loss

$$\ell(h(x), y) = \begin{cases} 0 & h(x) = y \\ 1 & h(x) \neq y \end{cases}$$

- Cross-Entropy Loss (used with probabilistic classifiers)

$$\ell(h(x), y) = -\log P(y|x)$$

Regression

- Squared Loss (Mean Squared Error)

$$\ell(h(x), y) = (h(x) - y)^2$$

- Absolute Loss

$$\ell(h(x), y) = |h(x) - y|$$

Supervised Learning Setup

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Input: A finite data set generated from an unknown distribution $(x, y) \sim \mathcal{D}$ over inputs and labels, and a hypothesis class H

The data set is assumed to be sampled **independently and identically distributed** from some underlying distribution $\mathcal{D}$.

Optimality: a hypothesis that minimizes the expected loss:

$$h^\star = \arg\min_{h \in H} \mathbb{E}_{(x,y)\sim\mathcal{D}} \left[\ell(h(x), y)\right]$$

Empirical Objective: Since the true distribution $\mathcal{D}$ is unknown, we approximate this objective using the empirical loss:

$$\hat{h} = \arg\min_{h \in H} \frac{1}{m} \sum_{i=1}^{m} \ell(h(x_i), y_i)$$

Supervised Learning Setup

In practice, we split the available dataset into three parts:

Training set S_{train}:

- used to **learn the model parameters**
- optimize empirical risk:

Validation (evaluation) set S_{val}:

- used to **tune hyperparameters** (e.g., model size)
- helps detect **overfitting**

Test set S_{test}:

- used **only once at the end**
- provides an **unbiased estimate of generalization performance**

Never use test data during training or model selection.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Supervised Learning Algorithms

There are several approaches to finding (or approximating) $\hat{h}$:

- **Analytical solutions** Closed-form minimization (e.g., linear regression with squared loss)
- **Iterative optimization** Gradient-based methods (e.g., SGD) for optimizing differentiable models, and search-based or gradient-free methods for non-differentiable models
- **Structured / symbolic methods** Search over hypothesis space (e.g., decision trees, rule learning)
- **Kernel / instance-based methods** Implicit hypothesis spaces (e.g., SVMs, nearest neighbors)
- **Pretrained / foundation models** Start from a large pretrained model and adapt:
  - fine-tuning on S
  - prompting / in-context learning

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

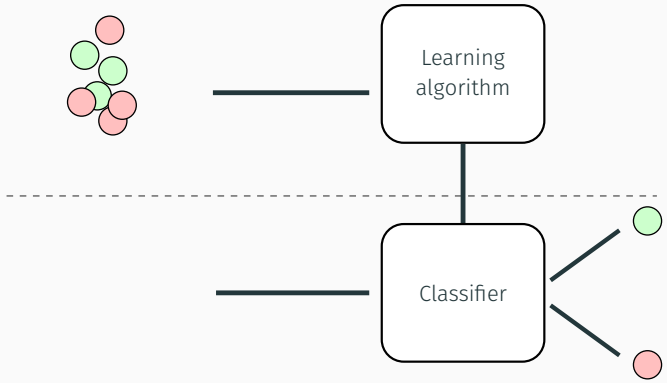
Example Approach: Iterative Supervised Learning via ERM

Algorithm 1 Iterative Supervised Learning via ERM

Require: Training set $S = \{(x_i, y_i)\}_{i=1}^{m}$, hypothesis class H, loss ℓ

Ensure: Learned hypothesis $\hat{h}$

1: Initialize hypothesis $h \in H$
2: **repeat**
3: Update h using optimization (e.g., gradient descent)
4: **until** convergence
5: $\hat{h} \leftarrow h$
6: **Return** $\hat{h}$



Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Example Update Rule: Gradient Descent

A common choice for the update function is **gradient descent**, which improves the current hypothesis by moving it in the direction that reduces the empirical risk (loss).

If we let the empirical risk be $\hat{\mathcal{R}}(h) = \frac{1}{m}\sum_{i=1}^{m} \ell(h(x_i), y_i)$, each iteration becomes:

$$h \leftarrow h - \eta \nabla \left(\frac{1}{m} \sum_{i=1}^{m} \ell\big(h(x_i), y_i\big) \right)$$

where:

- $\eta > 0$ is the **learning rate**
- $\nabla \hat{\mathcal{R}}(h)$ is the gradient of the empirical risk with respect to the model parameters

Gradient descent repeatedly adjusts the model parameters to reduce the training error.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Example Update Rule: Gradient Descent

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

In modern methods, deep learning is often used.
We parameterize the hypothesis as $h_\theta \in H$ with parameters θ.
The empirical risk is: $\hat{\mathcal{R}}(\theta) = \frac{1}{m}\sum_{i=1}^{m} \ell\big(h_\theta(x_i), y_i\big)$ and each iteration is:

$$\theta \leftarrow \theta - \eta \nabla_\theta \left(\frac{1}{m} \sum_{i=1}^{m} \ell\big(h_\theta(x_i), y_i\big) \right)$$

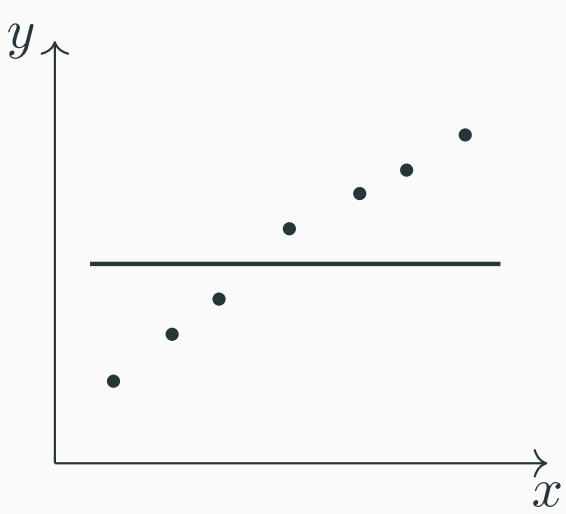
.

We iteratively update the parameters to reduce the empirical risk until convergence.

Supervised Learning : Key challenges

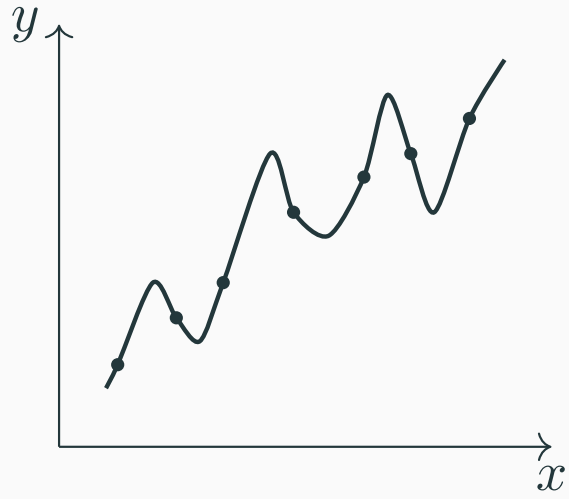
Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Underfitting



Avoid **underfitting**: using a model that is too simple to capture patterns.

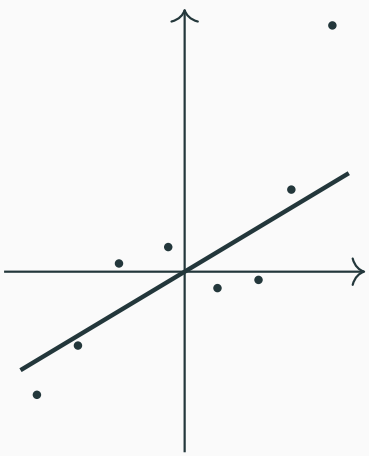
Overfitting



Avoid **overfitting**: fitting the training data too closely while failing to generalize.

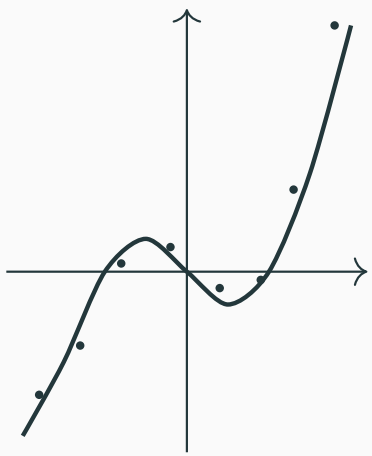
Example: Choosing Model Complexity

Assume the data is generated from an unknown cubic relationship: $y = x^3 - x + \epsilon$ where ϵ is a small noise term.



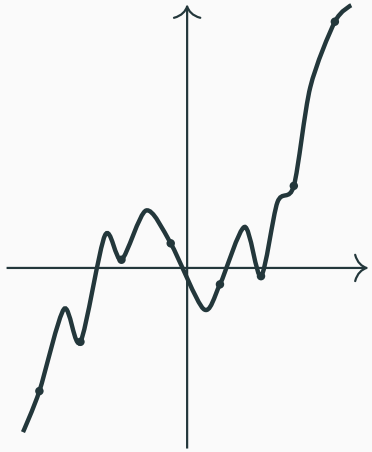
Fit with a **linear model**:

Too simple to capture the cubic trend.



Fit with a **degree-3 polynomial**:

Captures the underlying structure well.



Fit with a **n-degree polynomial**:

Fits the noise rather than the true pattern.

The goal is to find a model complexity that **generalizes well**.

Supervised Learning: Then vs. Now

Classical Supervised Learning

- Fixed dataset $S = \{(x_i, y_i)\}_{i=1}^{m}$
- Labels provided by humans or trusted sources
- Limited by the size and cost of labeled data
- Goal: learn $h \in H$ that generalizes to $\mathcal{D}$

Modern Setting with Foundation Models

- Large-scale pretrained models provide representations
- Unlabeled data can be automatically labeled (pseudo-labels)
- Data can be generated or augmented by models

Key Shift: We move from learning from labeled data to learning from data labeled by models. However, labels may be noisy or biased

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Supervised Learning as World-Model Learning

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

In many AI systems, supervised learning is used to learn and improve a model of the world.

World models learn predictive representations of the environment, modeling how observations, states, or latent variables evolve over time.

They range from deterministic predictors to probabilistic models (e.g., state-space and latent variable models) that capture uncertainty and support planning and reinforcement learning.

Supervised Learning as World-Model Learning

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

World models can be designed to support different types of predictions:

- predict object categories from sensor observations
- predict the next state given the current state and action (i.e., learn system dynamics)
- predict system failures from measurements
- infer hidden or latent properties from observations

These predictive models can be used to support planning and decision-making.

Foundation Models as Policies

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Recent AI systems increasingly use **foundation models directly as decision-making policies**.

Instead of only predicting the world, these models can map observations and instructions directly to actions.

Examples include:

- Vision-Language-Action (VLA) models for robotics
- Large Language Models (LLMs) used as agents
- Multimodal embodied agents

Foundation Models as Policies

VLA models combine:

- **Vision** - understanding sensor observations
- **Language** - interpreting goals and instructions
- **Action** - generating control commands

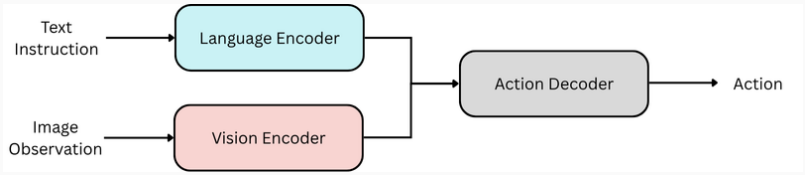


Figure 1: From wikipedia

These models are often trained from large datasets of demonstrations: $(observation_t, instruction_t) \rightarrow action_t$

This paradigm is closely related to imitation learning and behavioral cloning.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

From Supervised Learning to Decision-Making

Supervised learning (and ERM in particular) relies on the assumption that data is i.i.d. from some underlying distribution $\mathcal{D}$.

Implication 1: Distribution-specific guarantees

Even if we learn a model with low generalization error, the guarantees hold **only for the distribution** $\mathcal{D}$ from which the data was sampled.

Implication 2: Limits for decision-making

When predictions are used for **decision-making** or **interaction with an environment**, this becomes critical:

- The data distribution during deployment may differ from $\mathcal{D}$
- There are **no guarantees** that performance will remain good under such shifts

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

From Supervised Learning to Decision-Making

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

How does supervised learning relate to decision-making?

From Supervised Learning to Decision-Making

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

How does supervised learning relate to decision-making?

Supervised learning assumes a **fixed dataset**, while decision-making involves **interaction** with the environment.

We will explore two key connections:

- **Behavior cloning (imitation learning):** learn a policy from **expert demonstrations**, treating actions as labels.
- **Active learning:** **select** which data to acquire.

Behavior Cloning

Behavioral Cloning

Imitation learning is a family of methods where an agent learns to act by observing an expert.

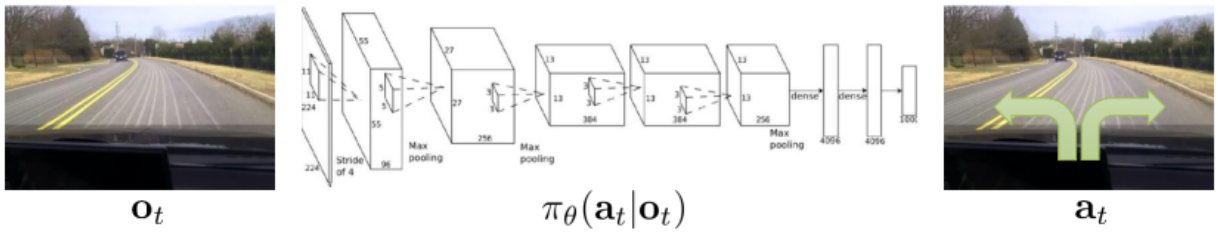
Behavior cloning is the simplest form of imitation learning, where the objective is to learn a policy by mimicking the behavior demonstrated by an expert.

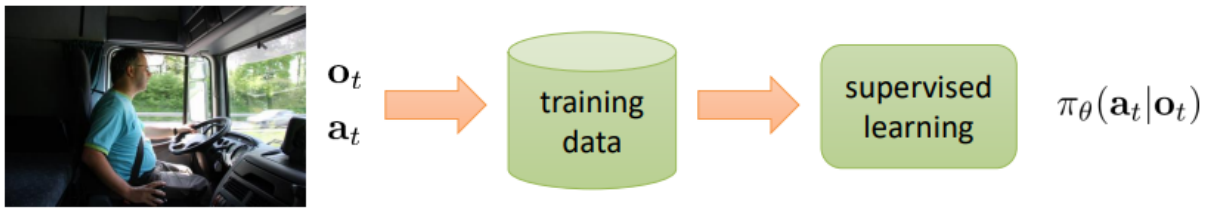
A policy is trained to minimize the discrepancy between its predicted actions and the expert's actions over the collected dataset.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Behavior Cloning: Example

Supervised Learning
Behavior Cloning
Active Learning
Conclusion





Behavioral cloning

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Can be broken down into the following steps:

- **Data Collection:** Gather a dataset of state-action pairs (s_i, a_i), where s_i represents the state and a_i represents the action taken by the expert at that state.
- **Model Selection:** Choose a model to approximate the expert's policy. Typically, a neural network $\pi_\theta(s)$ parameterized by θ.

Ideas for managing the learning process?

Behavioral Cloning

Loss: measures the difference between the expert's actions and the actions predicted by the model. For example, mean squared error (MSE) or cross-entropy loss for discrete actions leads to the following empirical risk:

$$\hat{\mathcal{R}}(\theta) = \frac{1}{m} \sum_{i=1}^{m} \|a_i - \pi_\theta(s_i)\|^2$$

where N is the number of state-action pairs in the dataset.

Training: Optimize the model parameters θ by minimizing the empirical risk (e.g., using gradient descent)

$$\theta^* = \arg\min_\theta \hat{\mathcal{R}}(\theta)$$

Policy Execution: Use the trained model $\pi_{\theta^*}(s)$ as the policy to predict actions for new states during execution.

Strengths? Limitations?

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Strengths of Behavioral Cloning

- **Simple and intuitive:** reduces policy learning to standard supervised learning
- **Data-driven:** can learn directly from expert demonstrations without specifying a reward function
- **Efficient training:** can leverage existing optimization methods and large supervised datasets
- **Useful when expert behavior is available:** especially effective when demonstrations are abundant and high quality
- **Good initialization:** often provides a strong starting point for further policy improvement

Behavior cloning makes sequential decision-making accessible through standard supervised learning tools.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Supervised Learning Assumption

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

We learn a hypothesis by minimizing empirical risk:

$$\hat{h} = \arg\min_{h \in H} \frac{1}{m} \sum_{i=1}^{m} \ell(h(x_i), y_i)$$

Key assumption: data is sampled i.i.d. from a fixed distribution

$$(x, y) \sim \mathcal{D}_{\text{train}} = \mathcal{D}_{\text{test}}$$

Implication

Generalization guarantees hold only under this assumption.

Limitations of Behavioral Cloning

In many settings, the IID assumption breaks because:

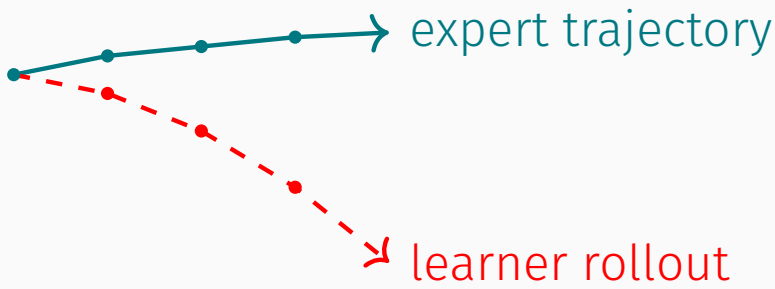
- Decisions are sequential: predictions are used to make decisions, which affect future inputs.
- The data distribution changes, i.e.,

$$\mathcal{D}_{\text{train}} \neq \mathcal{D}_{\text{test}}$$

The difference between the training and test distributions is called: distribution shift

- Train on one distribution, deploy on another
- Model encounters inputs it has never seen before
- Generalization guarantees no longer apply

Limitations of Behavioral Cloning



Distribution Shift: The agent follows π_θ trained on (s, a) from expert demonstrations, but encounters states that were never demonstrated.

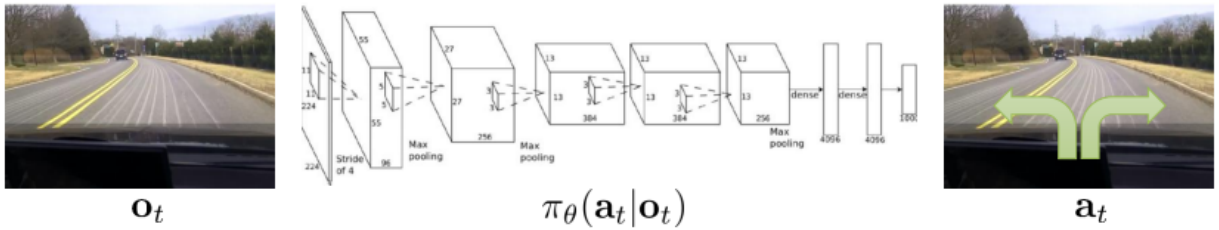
Key challenge: Errors compound over time.

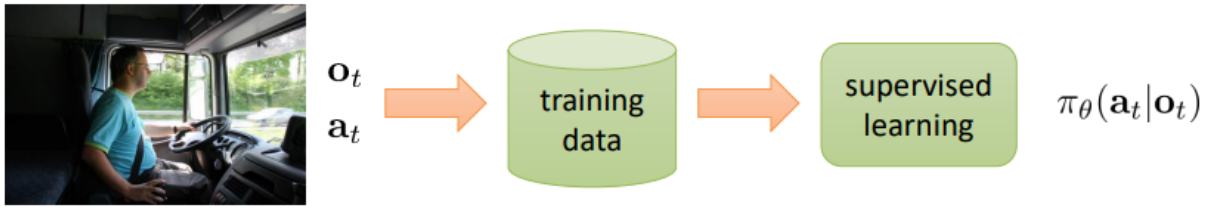
- Small prediction error leads to a slightly wrong action
- The system transitions to a new, unseen state
- Errors increase as the agent moves further from expert behavior

Effect: performance degrades rapidly.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Limitations of Behavioral Cloning





In autonomous driving, a small steering mistake moves the car off the lane, a situation rarely observed in expert data.

DAgger: Addressing the Distribution Shift

Key idea: train on the states the learner actually visits

- Run current policy π_i to collect states $s \sim \mathcal{D}^{\pi_i}$
- Query expert for correct actions a^*
- Aggregate dataset:

$$\mathcal{S} \leftarrow \mathcal{S} \cup \{(s, a^*)\}$$

- Retrain policy on updated dataset

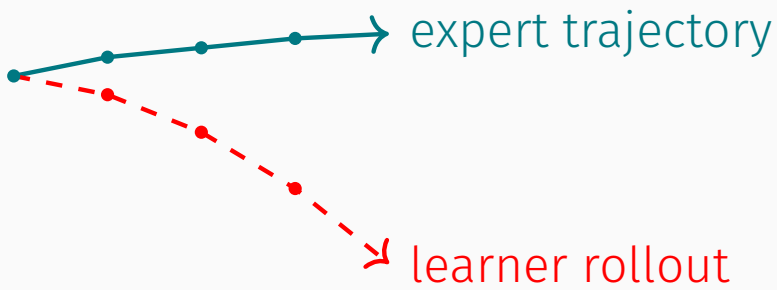
DAgger (Dataset Aggregation)

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Algorithm 2 Dataset Aggregation (DAgger)

1: Initialize dataset $\mathcal{D} = \{(o_1, a_1), \ldots, (o_N, a_N)\}$
2: **for** iterations **do**
3: Train policy $\pi_\theta(a_t \mid o_t)$ on $\mathcal{D}$
4: Collect observations $\{o_1, \ldots, o_M\}$ using π_θ
5: Query expert for labels $\{a_1^*, \ldots, a_M^*\}$
6: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(o_1, a_1^*), \ldots, (o_M, a_M^*)\}$
7: **end for**

DAgger



DAgger queries the expert on states visited by the learner, and adds these corrections to the training set.

What assumptions are made here about access to the expert during training?

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Limitations of Behavioral Cloning

Is this the same observation?

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Limitations of Behavioral Cloning

Is this the same observation?

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Limitations of Behavioral Cloning

Is this the same observation?

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Limitations of Behavioral Cloning

Is this the same observation?

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

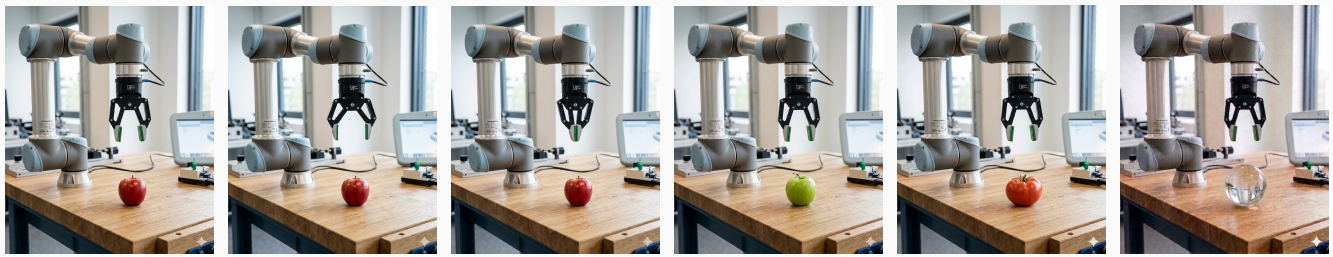
Limitations of Behavioral Cloning

Is this the same observation?

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

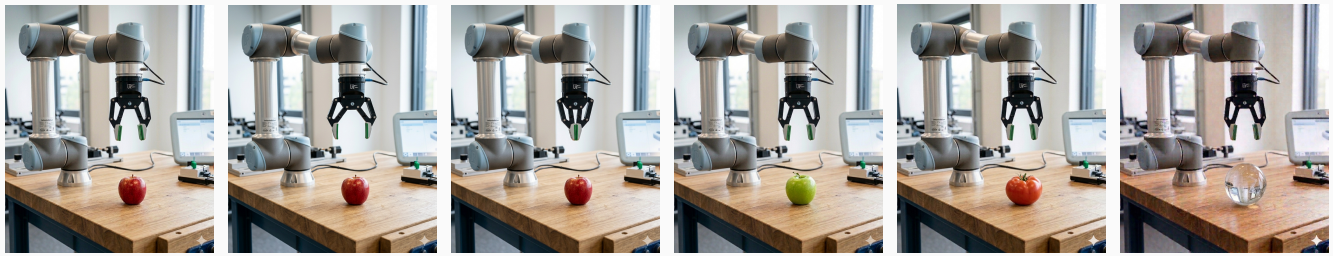
Limitations of Behavioral Cloning

Supervised Learning
Behavior Cloning
Active Learning
Conclusion



BC assumes a consistent mapping from observations to actions - this breaks under ambiguity and distribution shift.

Limitations of Behavioral Cloning



State aliasing \Perceptual aliasing (POMDP view): different underlying states produce visually similar observations, but require different actions:

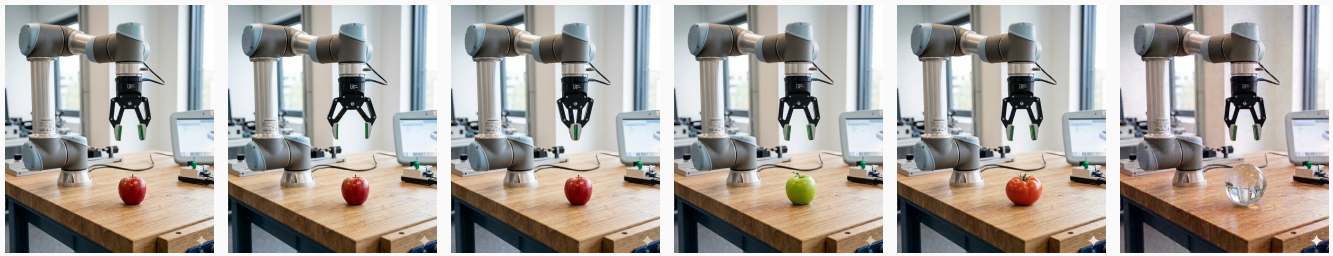
$$o \approx o' \quad \text{but} \quad a^\star \neq a^{\star\prime}$$

In behavior cloning, the policy is learned as a direct mapping from observations to actions, i.e., $\pi(a \mid o)$.

In this case, conflicting supervision is recevied and classical methods tend to learn the **average action**: $\mathbb{E}[a \mid o]$ which can be suboptimal or even invalid.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Limitations of Behavioral Cloning



The opposite can also occur: The same underlying state may appear visually different, and therefore be treated as out-of-distribution.

Covariate shift \Observation variability: The same state appears differently across contexts:

$$s = s' \quad \text{but} \quad o \neq o'$$

→ *observations fall outside training distribution*

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Behavior Cloning: Key Takeaways

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Behavior Cloning works well when:

- Observations are **informative** (no state aliasing)
- The action distribution is **unimodal** (a single action per state)
- Train and test distributions are **aligned**

Behavior Cloning often fails with:

- **Distribution shift:** $\mathcal{D}_{\text{train}} \neq \mathcal{D}_{\text{test}}$
- **State aliasing:** $o \approx o'$ but $a^\star \neq a^{\star\prime}$
- **Multimodality:** $\pi(a \mid o)$ has multiple valid modes

Key insight: Behavior cloning reduces decision-making to supervised learning — but **sequential structure and partial observability break its assumptions.**

This motivates interactive and sequential methods (e.g., DAgger, RL, ...).

Active Learning

Acknowledgment

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Some of the material is by Sang Truong (Stanford)

From Passive to Active Learning

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Standard supervised learning is passive.

- The dataset is given in advance
- The learner does not choose which examples appear
- The learner does not choose which labels are acquired

In many domains, labeled samples are expensive or rare.

- human annotation requires time and expertise
- experiments may be costly
- inspections may require physical access
- high-quality measurements may consume resources

From Passive to Active Learning

Examples:

- annotating medical images by doctors
- labeling road scenes for autonomous driving
- performing laboratory experiments
- collecting high-quality sensor measurements




Which example should we label next?

This question leads from **supervised learning** to **active learning**.

Active Learning

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Two primary setups:

- **Pool-based:** The model selects samples from a large unlabeled pool of data.
  For example, the most uncertain texts from a large pool to ask a human annotator to label.
- **Stream-based:** The model receives samples sequentially (one sample at a time) and decides whether to label them. The data is gone if the decision maker decides not to label it.
    - For example, a system monitoring sensor data decides on-the-fly whether new sensor readings are valuable enough to label.

We will focus on the former, but many of the insights and decision-making principles apply to both settings.

Active Learning

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Active learning allows the learner to **actively select** the most informative examples to label, based on information it can acquire using a **limited budget**.

Given an unlabeled pool of samples $S = \{x_1, \ldots, x_n\}$ the learner repeatedly:

1. selects a query $x^\star$
2. obtains its label $y^\star$
3. updates the model

Goal: maximize model accuracy given a limited labeling budget.

Active Learning is Not Control nor Meta-Learning

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

In control and reinforcement learning, actions are chosen to **affect the environment** and maximize cumulative reward.

In active learning, the learner chooses **which information to acquire**, rather than **which control action to execute**. Under the standard formulation, queries do not alter the environment state or data distribution.

In meta-learning, the goal is to **learn how to learn across tasks**, i.e., to acquire an **inductive bias** that enables rapid adaptation to new tasks.

In contrast, active learning focuses on a **single task**, aiming to **improve performance** by selecting informative data, rather than improving the learning procedure itself.

Active Learning is Sequential

Active learning unfolds over time.

At round t:

- labeled dataset: S_t
- unlabeled pool: U_t
- current model: $\Pr_{\theta_t}(y \mid x)$, trained on S_t, parametrized by θ_t

The learner selects:

$$x_t^\star = \arg\max_{x \in U_t} \mathrm{QueryScore}(x; \theta_t)$$

Receives label $y_t^\star$, and updates:

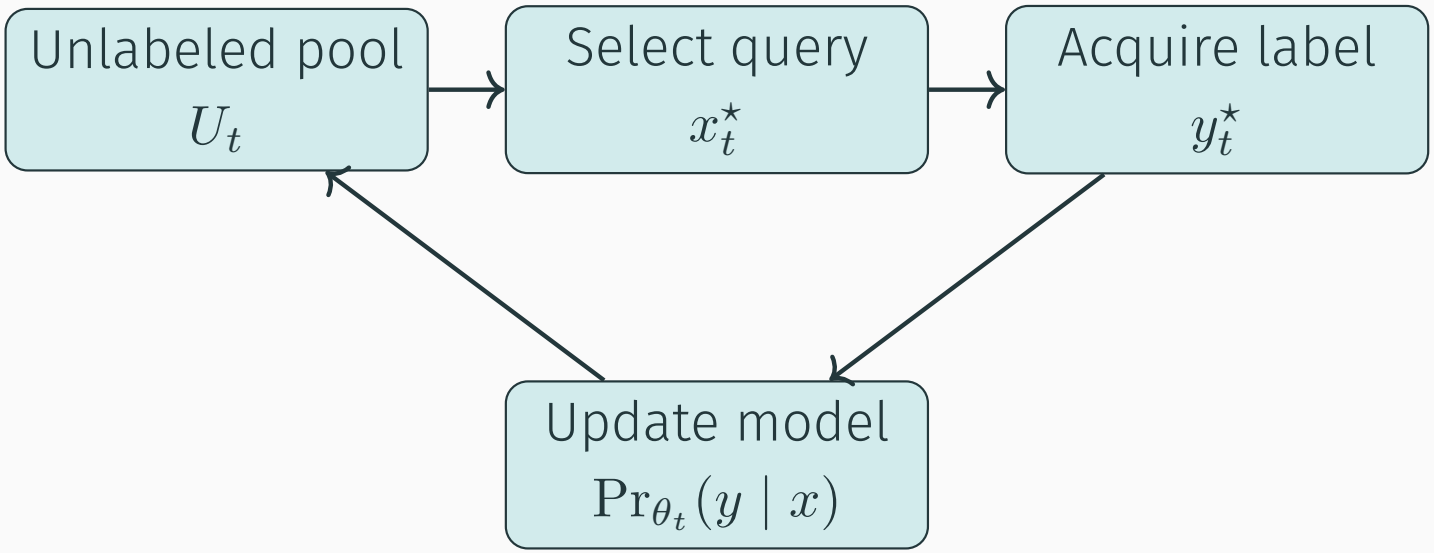
$$S_{t+1} = S_t \cup \{(x_t^\star, y_t^\star)\}$$

$$U_{t+1} = U_t \setminus \{x_t^\star\}$$

$$\theta_{t+1} \leftarrow \mathrm{Train}(S_{t+1})$$

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Active Learning Loop



The hypothesis space over the underlying model is represented by θ.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Active Learning

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Does active learning aim to address epistemic or aleatoric uncertainty?

Active Learning

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Does active learning aim to address epistemic or aleatoric uncertainty?

Epistemic uncertainty (model uncertainty) - can be reduced with more observations

Aleatoric uncertainty (inherent noise) - cannot be reduced by querying more data

Active learning is designed to prefer queries that reduce **epistemic uncertainty** and avoid queries dominated by **aleatoric noise**

Choosing the Next Query

Active learning selects queries that maximize the expected value of information.

Let $\mathcal{U} \subseteq \mathcal{X}$ is the pool of unlabeled examples.

$$x_{\text{next}} = \arg\max_{x \in \mathcal{U}} \text{QueryScore}(x; \theta)$$

Different strategies correspond to different definitions of $\text{QueryScore}(.)$ Common strategies include:

- **Uncertainty sampling:** query the point for which the current model is least certain
- **Query-by-committee:** query points on which several candidate models disagree
- **Expected error reduction:** query the point expected to most reduce future prediction error
- **Expected information gain:** query the point expected to most reduce uncertainty about the model

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Reminder: Entropy

Measures the **uncertainty** or **information content** of a random variable. Let X be a discrete random variable taking values in $\mathcal{X}$, distributed according to $p(x) := \Pr(X = x)$

$$\mathcal{H}(X) = \mathbb{E}\big[-\log p(X)\big]$$

Explicitly,

$$\mathcal{H}(X) = -\sum_{x} p(x) \log p(x) \qquad \text{(discrete case)}$$

$$\mathcal{H}(X) = -\int p(x) \log p(x)\, dx \qquad \text{(continuous / differential entropy)}$$

Interpretation for the discrete case: Expected number of bits required to identify the outcome of X.

Conditional entropy:

$$\mathcal{H}(Y \mid X) = \mathbb{E}\big[\mathcal{H}(Y \mid X = x)\big]$$

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Reminder: Information Gain

Information gain measures the reduction in uncertainty about one variable after observing another, i.e., how much knowing X reduces uncertainty about Y.

$$I(Y;X) = \mathcal{H}(Y) - \mathcal{H}(Y \mid X)$$

- $\mathcal{H}(Y)$: uncertainty about Y before observing X
- $\mathcal{H}(Y \mid X)$: remaining uncertainty after observing X

Equivalent symmetric form: $I(Y;X) = \mathcal{H}(X) - \mathcal{H}(X \mid Y)$

Alternative expression: $I(Y;X) = \mathcal{H}(Y) + \mathcal{H}(X) - \mathcal{H}(X,Y)$

Choosing the Next Query

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Uncertainty sampling: Query the point for which the current model, defined by θ, is least certain,

$$\mathrm{QueryScore}(x; \theta) = \mathcal{H}\big(p_\theta(y \mid x)\big)$$

Here, uncertainty is measured by the entropy of the model's predictive distribution.

Choosing the Next Query

Example: Consider a binary classification problem with two classes y_1 and y_2. We have three samples x_1, x_2, x_3 and the corresponding predictive distributions are as follows:

$$p(y_1|x_1) = 0.6, \quad p(y_2|x_1) = 0.4$$
$$p(y_1|x_2) = 0.3, \quad p(y_2|x_2) = 0.7$$
$$p(y_1|x_3) = 0.8, \quad p(y_2|x_3) = 0.2$$

Based on Entropy-based uncertainty sampling - which will be chosen next?

Choosing the Next Query

Example: Consider a binary classification problem with two classes y_1 and y_2. We have three samples x_1, x_2, x_3 and the corresponding predictive distributions are as follows:

$$p(y_1|x_1) = 0.6, \quad p(y_2|x_1) = 0.4$$
$$p(y_1|x_2) = 0.3, \quad p(y_2|x_2) = 0.7$$
$$p(y_1|x_3) = 0.8, \quad p(y_2|x_3) = 0.2$$

Based on Entropy-based uncertainty sampling - which will be chosen next?

- $\text{QueryScore}(x_1) = -0.6\log(0.6) - 0.4\log(0.4) = 0.29$
- $\text{QueryScore}(x_2) = -0.3\log(0.3) - 0.7\log(0.7) = 0.27$
- $\text{QueryScore}(x_3) = -0.8\log(0.8) - 0.2\log(0.2) = 0.22$

We would select x_1 for labeling as it has the highest entropy, indicating the model is most uncertain about its prediction at x_1.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Choosing the Next Query

Query-by-committee: Query points on which several candidate hypotheses disagree.

$$\text{QueryScore}(x) = \text{Var}_{h \in H}\left[h(x)\right] \quad \text{or} \quad \text{Disagreement}(x)$$

Maintain a **distribution or set of hypotheses** consistent with current data. Prefer queries where predictions vary the most across models

Where do different models come from?

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Choosing the Next Query

Query-by-committee: Query points on which several candidate hypotheses disagree.

$$\text{QueryScore}(x) = \text{Var}_{h \in H}\left[h(x)\right] \quad \text{or} \quad \text{Disagreement}(x)$$

Maintain a **distribution or set of hypotheses** consistent with current data. Prefer queries where predictions vary the most across models

Where do different models come from?

- **Ensembles:** different random initializations of optimization trajectories $\Rightarrow$ different local minima. Diversity emerges from non-convexity (e.g., neural networks).
- **Bootstrap resampling:** each model sees a slightly different dataset.
- **Bayesian posterior sampling:** over model parameters.

Approximating **epistemic uncertainty**
via disagreement between plausible models.

Choosing the Next Query

Expected error reduction: Query the point expected to most reduce future prediction error

$$\mathcal{R}(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{D}}\big[\ell(h_\theta(x), y)\big]$$

$$\text{QueryScore}(x) = \mathbb{E}_{y\sim \Pr_\theta(\cdot|x)}\left[\mathcal{R}(\theta) - \mathcal{R}\big(\theta^{+(x,y)}\big)\right]$$

- Measures expected improvement in generalization error
- This requires retraining for each candidate (x, y)
- In practice, the expected change is approximated, e.g., via local updates instead of full retraining, estimating change in gradients and decision boundary etc.information gain

Choosing the Next Query

Expected information gain: query the point expected to most reduce uncertainty about the model

$$\mathrm{QueryScore}(x) = I\big(y; \theta \mid x\big) = \mathcal{H}(\theta) - \mathcal{H}(\theta \mid y, x)$$

Where $\mathcal{H}(\theta) = -\int p(\theta) \log p(\theta)\, d\theta$

and $\mathcal{H}(\theta \mid y, x) = \mathbb{E}_{y \sim p(y|x)} \left[-\int p(\theta \mid y, x) \log p(\theta \mid y, x)\, d\theta \right]$

What is the connection to aleatoric and epistemic uncertainty?

Choosing the Next Query

Expected information gain:

$$I(y;\theta \mid x) = \underbrace{\mathcal{H}(p(y \mid x))}_{\text{total uncertainty}} - \underbrace{\mathbb{E}_\theta[\mathcal{H}(p(y \mid x,\theta))]}_{\text{aleatoric uncertainty}}$$

- Subtracts irreducible noise
- Focuses on uncertainty about the model

Good queries are those that are uncertain because we don't know the model (epistemic uncertainty)
not because the world is noisy (aleatoric uncertainty)

Choosing the Next Query in Continuous Domains

In continuous input spaces, we cannot evaluate $\arg\max_{x \in \mathcal{X}} \mathrm{QueryScore}(x)$ directly, since $\mathcal{X}$ is infinite.

Key idea: restrict the search to a finite candidate set $\mathcal{C} \subset \mathcal{X}$.

How do we construct $\mathcal{C}$?

- **Random sampling:** sample $x \sim p(x)$ from the input distribution
- **Pool-based setting:** select from a large unlabeled dataset $\mathcal{U}$
- **Grid / discretization:** uniform or adaptive discretization of the space
- **Local optimization:** directly optimize $\mathrm{QueryScore}(x)$ using gradient-based methods
- **Model-driven sampling:** focus on regions of high uncertainty (e.g., near decision boundary)

Active Learning with Gaussian Processes

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

We want to learn an unknown function $f(x)$ from noisy observations

$$y_i = f(x_i) + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

A Gaussian Process (GP) places a distribution over functions:

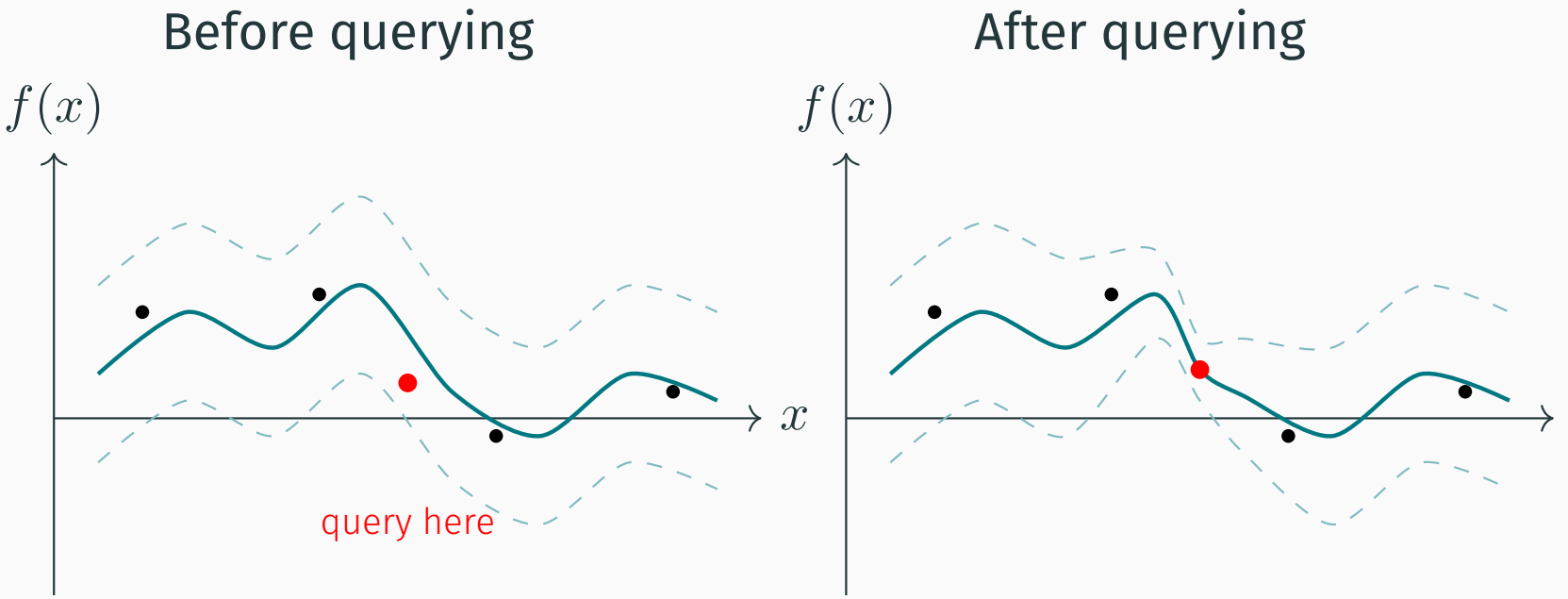
$$f(x) \sim \mathcal{GP}(m(x), k(x, x'))$$

where

- $m(x)$ is the mean function
- $k(x, x')$ is the kernel (covariance) function

Interpretation A GP defines a prior belief about possible functions that explain the model of the environment.

Active Learning with Gaussian Processes



- Querying a point reveals its value and updates the posterior over functions
- Uncertainty shrinks most strongly near the queried location
- Future queries are selected using the updated posterior

Key idea: each query changes the belief over functions, reducing uncertainty and guiding the next query.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Active Learning with Gaussian Processes

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

A Gaussian Process (GP) provides a **principled Bayesian model** that enables directly computing uncertainty and information gain

In particular, a GP provides for each x:

- Predictive mean: $\mu(x)$
- Predictive variance: $\sigma^2(x)$

This enables natural definitions of $\mathrm{QueryScore}(x)$:

- **Uncertainty sampling:** $\mathrm{QueryScore}(x) = \sigma^2(x)$
- **Information gain:** closed-form expressions for $I(y; f \mid x)$

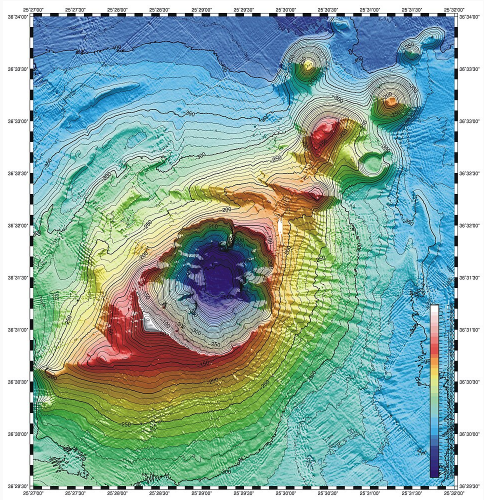
Example: Active Environmental Sensing

A robot wants to learn a spatial field $f(x)$ (e.g., temperature, radiation, terrain cost), but measurements are expensive.

Using a Gaussian Process:

- the GP predicts the field value at each location
- uncertainty indicates unexplored regions

Active learning decides where to measure next.



The robot learns the map efficiently with fewer measurements.

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Active learning as decision-making under uncertainty

Selecting a query x is analogous to taking an information-gathering action.

Connection to Planning and Reinforcement Learning

- Query selection $\leftrightarrow$ action selection
- Labels/observations $\leftrightarrow$ stochastic outcomes
- Model update $\leftrightarrow$ belief update
- Objective $\leftrightarrow$ maximize long-term utility

Active learning can be viewed as a myopic POMDP, where queries are actions and information has utility.

$$\text{VOI}(x) = \mathbb{E}_{y \sim p(y|x)}\left[U(\theta \mid \mathcal{D} \cup \{(x, y)\})\right] - U(\theta \mid \mathcal{D})$$

Conclusion

Conclusion

Supervised Learning
Behavior Cloning
Active Learning
Conclusion

Considering each approach to behavior cloning and active learning, we have seen:

- Is it **complete**? Is it **optimal**?
- What are its **time and space requirements**?
- Does it exhibit **anytime behavior**?
- How does it use **resources** (computation, memory, data)?
- Can it handle **uncertainty** or only deterministic settings?